

```
[ ]: ## Data Preprocessing
```

```
[1]: import pandas as pd
import seaborn as sns
#Load Titanic dataset
df=sns.load_dataset("titanic")
print(df.head())
print(df.shape)
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

(891, 15)

```
[2]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
```

```
[ ]: ## Data Preprocessing
```

```
[1]: import pandas as pd
import seaborn as sns
#Load Titanic dataset
df=sns.load_dataset("titanic")
print(df.head())
print(df.shape)
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

(891, 15)

```
[2]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
```

```
5  parch      891 non-null    int64
6  fare       891 non-null    float64
7  embarked   889 non-null    object
8  class      891 non-null    category
9  who        891 non-null    object
10 adult_male  891 non-null    bool
11 deck       203 non-null    category
12 embark_town 889 non-null    object
13 alive      891 non-null    object
14 alone      891 non-null    bool
```

dtypes: bool(2), category(2), float64(2), int64(4), object(5)

memory usage: 80.7+ KB

```
[ ]: # 1. Inconsistent formats
```

concept

the same value **is** represented **in** diffent formats. *# same daat different formats*

Create problem

```
[3]: df.loc[0,'embark_town'] = 'southampton'
df.loc[1,'embark_town'] = 'SOUTHAMPTON'
df.loc[2,'embark_town'] = 'Southampton'
print(df['embark_town'].unique())
```

```
['southampton' 'SOUTHAMPTON' 'Southampton' 'Queenstown' 'Cherbourg' nan]
```

```
[11]: df['embark_town'] = df['embark_town'].str.title()    # all the words 1st letter will be in uppercase
print(df['embark_town'].unique())                      #it will give only unique words
```

```
['Southampton' 'Queenstown' 'Cherbourg' nan]
```

```
[9]: print(df['embark_town'].isnull().sum())
```

```
2
```

```
[12]: df['embark_town'] = df['embark_town'].fillna('Kingstown')
print(df['embark_town'].unique())
```

```
[9]: print(df['embark_town'].isnull().sum())
```

```
2
```

```
[12]: df['embark_town'] = df['embark_town'].fillna('Kingstown')
```

```
print(df['embark_town'].unique())
```

```
['Southampton' 'Queenstown' 'Cherbourg' 'Kingstown']
```

```
[ ]: #2. Duplicate Rows
```

A duplicate row **is** an exact copy of an another row

Create duplicate records

```
[13]: duplicate_rows = df.iloc[:5]
```

```
df_dup = pd.concat([df,duplicate_rows])
```

```
print(df_dup.shape)
```

```
(896, 15)
```

```
[14]: #Find duplicates
```

```
print(df_dup.duplicated().sum())
```

```
112
```

```
[15]: #view duplicate rows
```

```
df_dup[df_dup.duplicated()]
```

```
[15]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
47	1	3	female	NaN	0	0	7.7500	Q	Third	woman	False	NaN	Queenstown	yes	True
76	0	3	male	NaN	0	0	7.8958	S	Third	man	True	NaN	Southampton	no	True
77	0	3	male	NaN	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True
87	0	3	male	NaN	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False		
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Southampton	yes	False		
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True		
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False		
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True		

112 rows × 15 columns

```
[18]: #Remove duplicates
df_dup=df_dup.drop_duplicates()
print(df_dup.duplicated().sum())
```

0

```
[28]: #3. Typos

#spelling mistakes create unnecessary categories
#create type

df.loc[0,'embark_town'] = 'Southampton'
df.loc[1,'embark_town'] = 'Southampton'
print(df['embark_town'].value_counts())
```

```
embark_town
Southampton    643
Cherbourg      167
Queenstown     77
Kingstown       2
Southampton     1
```

```

Queenstown      77
Kingstown        2
Southampton      1
Southampton      1
Name: count, dtype: int64

```

[29]: *#Fix typos*

```

df['embark_town']=df['embark_town'].replace({
    'Southamton':'Southampton',
    'Southhampton':'Southampton'
})

#Verify
print(df['embark_town'].value_counts()) #back to form

```

```

embark_town
Southampton    645
Cherbourg      167
Queenstown     77
Kingstown       2
Name: count, dtype: int64

```

[30]: `print(df['embark_town'].unique())`

```
['Southampton' 'Queenstown' 'Cherbourg' 'Kingstown']
```

• [31]:

```

print(sorted(df['embark_town'].dropna().unique()))

['Cherbourg', 'Kingstown', 'Queenstown', 'Southampton']

```

[]: *#4. Unit Mismatches*

values representing the same measurement stored **in** diff units

create problems

Suppose some ages are accidentally entered **in** months instead of years.

```
[35]: df.loc[0, 'age'] = 240
      df.loc[1, 'age'] = 300
```

```
[36]: #Detect unusual values
      df['age'].describe()
```

```
[36]: count    714.000000
      mean     30.371387
      std     19.362012
      min      0.420000
      25%     20.125000
      50%     28.000000
      75%     38.750000
      max     300.000000
      Name: age, dtype: float64
```

```
*[37]: # Fix unit mismatch
      #convert months to years
      df.loc[df['age'] > 120, 'age']=(
          df.loc[df['age'] > 120, 'age'] / 12      # change from month to year
      )

      #verify
      df['age'].describe()
```

```
[37]: count    714.000000
      mean     29.678109
      std     14.525902
      min      0.420000
      25%     20.000000
      50%     28.000000
      75%     38.000000
      max      80.000000
      Name: age, dtype: float64
```


Name: age, dtype: float64

```
[ ]: # 5. Wrong category labels
```

```
df['survived']
```

```
contains:
```

```
0
```

```
1
```

```
Not very readable
```

```
[38]: df['survived_label'] = df['survived'].map({
```

```
    0: 'No',
```

```
    1: 'Yes'
```

```
})
```

```
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	20.0	1	0	7.2500	S	Third
1	1	1	female	25.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone	survived_label
0	man	True	NaN	Southampton	no	False	No
1	woman	False	C	Southampton	yes	False	Yes
2	woman	False	NaN	Southampton	yes	True	Yes
3	woman	False	C	Southampton	yes	False	Yes
4	man	True	NaN	Southampton	no	True	No

```
[ ]: # Reduntant columns
```

```
columns with little or no useful information.
```

```
Example:
```

```
df['deck']
```

```
contains mostly missing values.
```


3	woman	False	C	Southampton	yes	False	Yes
4	man	True	NaN	Southampton	no	True	No

```
[ ]: # Reduntant columns
columns with little or no useful information.
```

```
Example:
df['deck']
contains mostly missing values.
```

```
*[40]: #check
df.isnull().mean() #to find null values
```

```
[40]: survived      0.000000
pclass      0.000000
sex          0.000000
age          0.198653
sibsp        0.000000
parch        0.000000
fare         0.000000
embarked     0.002245
class        0.000000
who          0.000000
adult_male   0.000000
deck         0.772166
embark_town  0.000000
alive        0.000000
alone        0.000000
survived_label 0.000000
dtype: float64
```

```
[41]: #Remove
df.drop(columns=['deck'],inplace=True)
df.head()
```

```
[41]: survived pclass sex age sibsp parch fare embarked class who adult_male embark_town alive alone survived_label
```

[41]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived_label
0	0	3	male	20.0	1	0	7.2500	S	Third	man	True	Southampton	no	False	No
1	1	1	female	25.0	1	0	71.2833	C	First	woman	False	Southampton	yes	False	Yes
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	Yes
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False	Yes
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True	No

[]: *#Handling null values in titanic dataset*

what are null values?
 null values (missing values) occur when data is unavailable

[43]: *#check Missing values*
`df.isnull().sum()`

```
[44]: #Method 1 : Mean Imputation
```

```
# Replace missing values with the average value
df['age'] = df['age'].fillna(df['age'].mean())
df.head(20)
```

```
[44]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived_label
0	0	3	male	20.000000	1	0	7.2500	S	Third	man	True	Southampton	no	False	No
1	1	1	female	25.000000	1	0	71.2833	C	First	woman	False	Southampton	yes	False	Yes
2	1	3	female	26.000000	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	Yes
3	1	1	female	35.000000	1	0	53.1000	S	First	woman	False	Southampton	yes	False	Yes
4	0	3	male	35.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True	No
5	0	3	male	29.678109	0	0	8.4583	Q	Third	man	True	Queenstown	no	True	No
6	0	1	male	54.000000	0	0	51.8625	S	First	man	True	Southampton	no	True	No
7	0	3	male	2.000000	3	1	21.0750	S	Third	child	False	Southampton	no	False	No
8	1	3	female	27.000000	0	2	11.1333	S	Third	woman	False	Southampton	yes	False	Yes
9	1	2	female	14.000000	1	0	30.0708	C	Second	child	False	Cherbourg	yes	False	Yes
10	1	3	female	4.000000	1	1	16.7000	S	Third	child	False	Southampton	yes	False	Yes
11	1	1	female	58.000000	0	0	26.5500	S	First	woman	False	Southampton	yes	True	Yes
12	0	3	male	20.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True	No
13	0	3	male	39.000000	1	5	31.2750	S	Third	man	True	Southampton	no	False	No
14	0	3	female	14.000000	0	0	7.8542	S	Third	child	False	Southampton	no	True	No
15	1	2	female	55.000000	0	0	16.0000	S	Second	woman	False	Southampton	yes	True	Yes

14	0	3	female	14.000000	0	0	7.8342	S	Third	child	False	Southampton	no	True	No
15	1	2	female	55.000000	0	0	16.0000	S	Second	woman	False	Southampton	yes	True	Yes
16	0	3	male	2.000000	4	1	29.1250	Q	Third	child	False	Queenstown	no	False	No
17	1	2	male	29.678109	0	0	13.0000	S	Second	man	True	Southampton	yes	True	Yes
18	0	3	female	31.000000	1	0	18.0000	S	Third	woman	False	Southampton	no	False	No
19	1	3	female	29.678109	0	0	7.2250	C	Third	woman	False	Cherbourg	yes	True	Yes

```
[49]: # Methos2 : Median Imputation
#Replace missing values with the middle value.
df['age'] = df['age'].fillna(df['age'].median())
df.head(20)
```

```
[49]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived_label
0	0	3	male	20.000000	1	0	7.2500	S	Third	man	True	Southampton	no	False	No
1	1	1	female	25.000000	1	0	71.2833	C	First	woman	False	Southampton	yes	False	Yes
2	1	3	female	26.000000	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	Yes
3	1	1	female	35.000000	1	0	53.1000	S	First	woman	False	Southampton	yes	False	Yes
4	0	3	male	35.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True	No
5	0	3	male	29.678109	0	0	8.4583	Q	Third	man	True	Queenstown	no	True	No
6	0	1	male	54.000000	0	0	51.8625	S	First	man	True	Southampton	no	True	No
7	0	3	male	2.000000	3	1	21.0750	S	Third	child	False	Southampton	no	False	No
8	1	3	female	27.000000	0	2	11.1333	S	Third	woman	False	Southampton	yes	False	Yes
9	1	2	female	14.000000	1	0	30.0708	C	Second	child	False	Cherbourg	yes	False	Yes

11	1	1	female	58.000000	0	0	26.5500	S	First	woman	False	Southampton	yes	True	Yes
12	0	3	male	20.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True	No
13	0	3	male	39.000000	1	5	31.2750	S	Third	man	True	Southampton	no	False	No
14	0	3	female	14.000000	0	0	7.8542	S	Third	child	False	Southampton	no	True	No
15	1	2	female	55.000000	0	0	16.0000	S	Second	woman	False	Southampton	yes	True	Yes
16	0	3	male	2.000000	4	1	29.1250	Q	Third	child	False	Queenstown	no	False	No
17	1	2	male	29.678109	0	0	13.0000	S	Second	man	True	Southampton	yes	True	Yes
18	0	3	female	31.000000	1	0	18.0000	S	Third	woman	False	Southampton	no	False	No
19	1	3	female	29.678109	0	0	7.2250	C	Third	woman	False	Cherbourg	yes	True	Yes

```
[ ]: let's check Age distribution
      #df['age'].skew()
      if skewness is high:
          Use median.
      if data is nearly normal:
          Use mean
```

```
[ ]: # Method 3: mode imputation
      Replace missing values with the most frequent value.
      #df['embark_town'] = df['embarked'].fillna(df['embarked'].mode()[0])
```

```
[ ]: #Method 4: Forward fill(FFill)
      use the previous val to fill missing data
      Before imputation
```

```
[50]: print(df['embarked'].isnull().sum())
```

2

```
[51]: #Apply forward Fill
df['embarked_ffill'] = df['embarked'].ffill()
#check missing values
print(df['embarked_ffill'].isnull().sum())
```

0

```
•[52]: df.columns          # embarked ffill added
```

```
[52]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
         'embarked', 'class', 'who', 'adult_male', 'embark_town', 'alive',
         'alone', 'survived_label', 'embarked_ffill'],
        dtype='object')
```

```
[ ]: #method5 : KNN Imputation
```

K- Nearest neighbours (KNN) imputation fills missing values using information from similar rows. Instead of using a single value(mean/median),it finds the K most similar records and uses their values

```
[53]: import pandas as pd
import numpy as np
data={
    'age':[22,25,np.nan,24,23],
    'fare':[7.25,8.05,7.90,8.00,7.50]
}
df=pd.DataFrame(data)
print(df)
```

	age	fare
0	22.0	7.25
1	25.0	8.05
2	NaN	7.90
3	24.0	8.00
4	23.0	7.50

```
3 24.0 8.00
4 23.0 7.50
```

```
[55]: #Apply KNN Imputation
from sklearn.impute import KNNImputer
KNN=KNNImputer(n_neighbors=3)
df_KNN = pd.DataFrame(
    KNN.fit_transform(df),
    columns=df.columns)
print(df_KNN)

#sklearn -> scikit-Learn Library for machine learning and data preprocessing
#impute -> Module for handling missing values
#KNNImputer -> class that fills missing values using the K-Nearest Neighbors(KNN)
#fit_transform(df) -> learns from the data and fills missing values
#converts the result back into a pandas DataFrame.
```

```
   age  fare
0  22.0  7.25
1  25.0  8.05
2  24.0  7.90
3  24.0  8.00
4  23.0  7.50
```

```
[59]: #Check missing values
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer

#Load Titanic Dataset
df=sns.load_dataset('titanic')

#Forward fill for embarked
df['embarked_ffill']=df['embarked'].ffill()
```


#KNN Imputation for age

```
cols=[
    'age',
    'fare',
    'sibsp',
    'parch',
    'pclass'
]

imputer=KNNImputer(n_neighbors=5)

df[cols]=imputer.fit_transform(df[cols])

print(df.isnull().sum())
```

```
survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town    2
alive         0
alone         0
embarked_ffill 0
dtype: int64
```

```
Backward fill
df.fillna(method='bfill',inplace=True)
or
df.bfill(inplace=True)
```

Encoding using titanic dataset

what is encoding?

Machine learning models work with numbers , not text.

Example:

sex

male

female

male

A model cannot directly understand "male" and "female".

We convert categories into numbers using encoding.

Types of Encoding

1.Label encoding(original data)

Assigns a number to each category.

Example:

Size

.Small

.Medium

.Large

becomes

Size

.0

.1

.2

When to Use Label Encoding

```
[ ]: ordinal categories
```

Example:

small<Medium<Large

Low<Medium<High

When to Use Label Encoding

```
[ ]: ordinal categories
Example:
small<Medium<Large
Low<Medium<High
Poor<Average<Good
```

Nominal Categories

```
[ ]: Example:
Red,Blue,Green
Male,Female
India,USA,UK
```

```
[ ]: #2. One-Hot Encoding(Nominal Data)
Creates a separate binary column for each category.
```

when to use One-Hot Encoding?

Nominal categories
Country,Gender,Color,City,Department

3.Target Encoding
Replace categories with the average value of the target variable.

Why Use Target Encoding?

Imagine:

5000 cities

One Hot Encoding would create:

5000 cities

One-Hot Encoding would create:

5000 columns

Not practical.

Target Encoding creates:

1 column

```

* [60]: import pandas as pd
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

#Load Titanic dataset
df = sns.load_dataset('titanic')

#Label Encoding
le = LabelEncoder()
df['sex_encoded'] = le.fit_transform(df['sex'])

# One-Hot Encoding
df = pd.get_dummies(df,
    columns=['embarked'], # drop_first = True this can be done at the last
)

#Target Encoding
target_map = df.groupby('pclass')['survived'].mean()
df['class_target'] = (df['pclass'].map(target_map))
print(df.head())

```

	survived	pclass	sex	age	sibsp	parch	fare	class	who	\
0	0	3	male	22.0	1	0	7.2500	Third	man	
1	1	1	female	38.0	1	0	71.2833	First	woman	
2	1	3	female	26.0	0	0	7.9250	Third	woman	
3	1	1	female	35.0	1	0	53.1000	First	woman	
4	0	3	male	35.0	0	0	8.0500	Third	man	

	adult_male	deck	embark_town	alive	alone	sex_encoded	embarked_C	\
0	True	NaN	Southampton	no	False	1	False	
1	False	C	Cherbourg	yes	False	0	True	
2	False	NaN	Southampton	yes	True	0	False	
3	False	C	Southampton	yes	False	0	False	
4	True	NaN	Southampton	no	True	1	False	

	embarked_Q	embarked_S	class_target
0	False	True	0.242363
1	False	False	0.629630
2	False	True	0.242363
3	False	True	0.629630
4	False	True	0.242363

Feature Scaling Using Titanic Dataset

```
[ ]: What is Feature scaling?
Feature Scaling transforms numerical features to a similar scale.

Why it is needed?
In the Titanic dataset

Example:
Feature Range Age 0-80 Fare 0-512
Here,Fare has much larger range than Age.
Many ML algorithms calculate distance,so Fare can dominate the model.
```

```
1 MinMaxScaler
```

1.MinMaxScaler

Scales values between 0 and 1.

Advantages :

Preserves shape of distribution

values always between 0 and 1

Useful fo Neural Networks

Disadvantages:

Sensitive to outliers

2.StandardScaler

Centers data around mean = 0 and standard deviation =1.

Advantages:

Works well for normally distributed data

Commonly used

Required by many algorithms

Best For

Logistics Regression,SVM,PCA,KNN,Linear Regression.

Disadvantages:

Still affected by extreme outliers

3.RobustScaler

Uses Median and IQR instead of Mean and Standard Deviation.

Advantages

Handles outliers well

Better for skewed distribution

Disadvantages

Values are not limited to 0-1

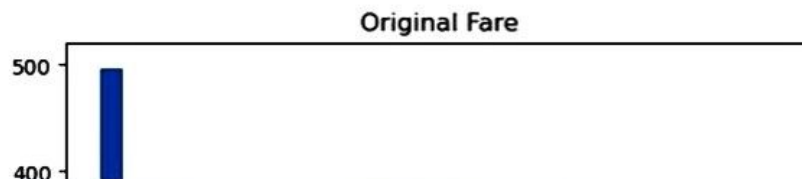
```
[61]: from sklearn.preprocessing import MinMaxScaler
      scaler = MinMaxScaler()
      df[['age_minmax', 'fare_minmax']] = scaler.fit_transform(
          df[['age', 'fare']]
      )
      df[['age_minmax', 'fare_minmax']].head()
```

```
[61]:
```

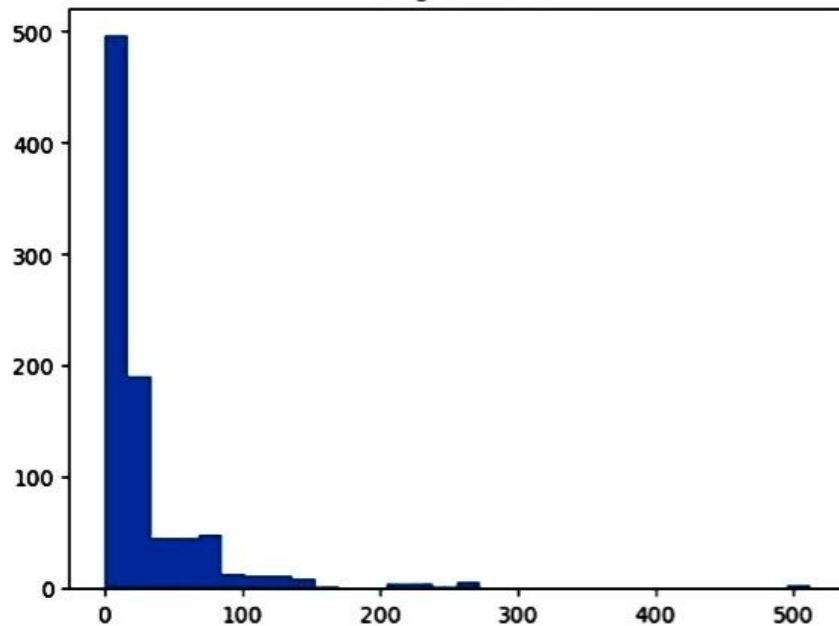
	age_minmax	fare_minmax
0	0.271174	0.014151
1	0.472229	0.139136
2	0.321438	0.015469
3	0.434531	0.103644
4	0.434531	0.015713

```
[62]: import matplotlib.pyplot as plt
      plt.hist(df['fare'].dropna(),bins=30)
      plt.title("Original Fare")
      plt.show()

      plt.hist(df['fare_minmax'],bins=30)
      plt.title("MinMax Scaled Fare")
      plt.show()
```

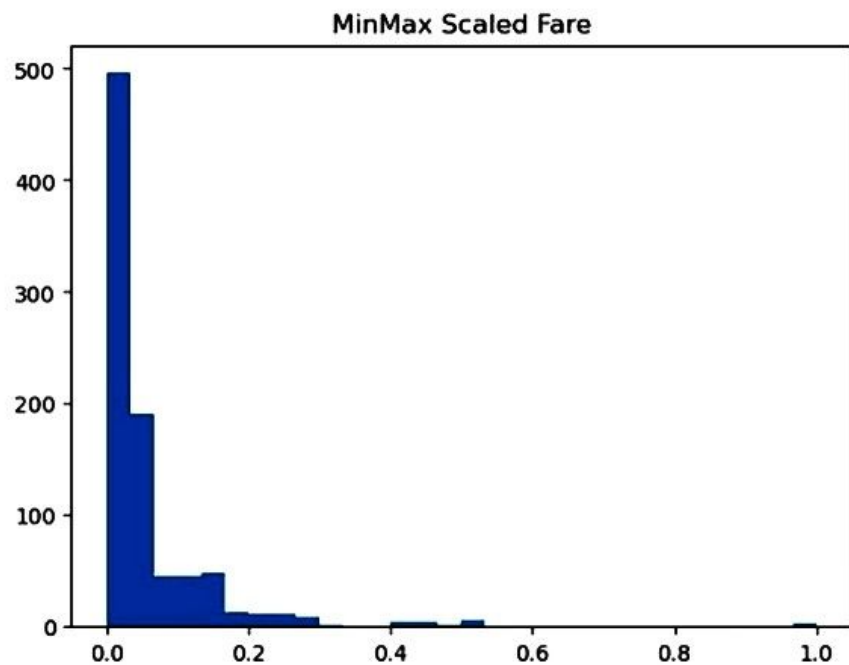


Original Fare



MinMax Scaled Fare





```
[63]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['age_std', 'fare_std']] = scaler.fit_transform(
    df[['age', 'fare']]
)
df[['age_std', 'fare_std']].head()
```

```
[63]: age_std  fare_std
```

[63]:

	age_std	fare_std
0	-0.530377	-0.502445
1	0.571831	0.786845
2	-0.254825	-0.488854
3	0.365167	0.420730
4	0.365167	-0.486337

```
[65]: from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
df[['age_robust', 'fare_robust']] = scaler.fit_transform(
    df[['age', 'fare']]
)
df[['age_robust', 'fare_robust']].head()
```

[65]:

	age_robust	fare_robust
0	-0.335664	-0.312011
1	0.559441	2.461242
2	-0.111888	-0.282777
3	0.391608	1.673732
4	0.391608	-0.277363

[]: